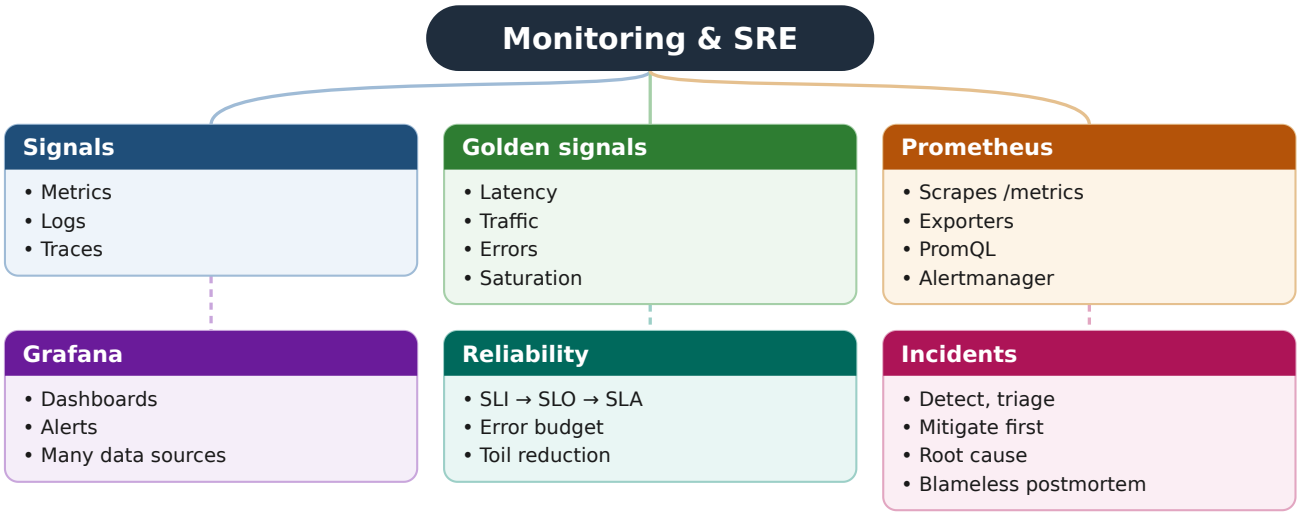
13. Monitoring & SRE

DevOps, Cloud & Platform Engineering · Complete Notes · Topic 13 of 18 · Thirupathi Keloth · v2.0

Contents

13.0 Why this matters
13.1 Fundamentals
13.2 Metrics in depth
13.3 Prometheus
13.4 PromQL in depth
13.5 Instrumenting an application
13.6 Alerting
13.7 Grafana
13.8 Logging
13.9 Distributed tracing and OpenTelemetry
13.10 AWS CloudWatch and native observability
13.11 Site Reliability Engineering
13.12 Incident management
13.13 Capacity, load testing and chaos engineering
13.14 A production monitoring stack (reference design)
13.15 Troubleshooting the monitoring system
13.16 Hands-on labs
13.17 Interview questions
13.18 Self-check



13.0 Why this matters

Shipping software is half the job; knowing **whether it works for users right now** is the other half. Without observability, you learn about outages from customers, you debug by guessing, and you can't tell whether a release made things better or worse. Site Reliability Engineering (SRE) adds the discipline on top: measurable reliability targets, alerting on what users feel, and a structured way to handle and learn from incidents.

13.1 Fundamentals

Term	Meaning
Monitoring	Collecting predefined measurements and alerting on known failure modes ("is CPU > 90%?", "is the site up?")

Term	Meaning
Observability	How well you can understand a system's internal state from its outputs, including questions you didn't predict ("why are only Android users in Mumbai slow since 14:05?")
Telemetry	The data a system emits: metrics, logs, traces (plus profiles and events)
Instrumentation	Code or agents that produce telemetry

The signals

Signal	What it is	Strength	Weakness	Tools
Metrics	Numeric time series with labels (`http_requests_total{status="500"}`)	Cheap, fast, ideal for dashboards and alerts, long retention	Little detail per event; label cardinality limits	Prometheus, CloudWatch, Datadog
Logs	Timestamped event records	Rich detail, exact errors	Expensive at volume, slow to aggregate	Loki, OpenSearch/ELK, CloudWatch Logs, Splunk
Traces	The path of one request across services (spans with timings)	Shows where latency/errors happen in distributed systems	Need instrumentation and sampling	OpenTelemetry, Jaeger, Tempo, X-Ray
Profiles	Where CPU/memory is spent in code over time	Finds hot code paths	Specialized	Pyroscope, Parca
Events	Deploys, config changes, scaling	Correlate changes with symptoms	Often missing	Annotations, change logs

Correlation is the goal: a dashboard spike (metric) → the traces for slow requests in that minute → the logs of the failing span (trace ID in the log line). Use the same labels everywhere (`service`, `env`, `version`, `region`).

What to measure: three standard methods

Method	For	Signals
Four Golden Signals (Google SRE)	Any user-facing service	**Latency** (separate successful from failed requests), **Traffic** (requests/s), **Errors** (rate of failures), **Saturation** (how "full": CPU, memory, queue depth, connection pools)
RED	Request-driven services (APIs)	**R**ate, **E**rrors, **D**uration
USE (Brendan Gregg)	Resources (CPU, disk, network, pools)	**U**tilization, **S**aturation, **E**rrors

Monitor **symptoms** (what users feel: errors, latency) for paging; use **causes** (CPU, disk, a pod restart) for dashboards and diagnosis.

Averages lie: percentiles

If 99 requests take 100 ms and 1 takes 10 s, the average is ~200 ms, which describes nobody. Use percentiles: **p50** (median, typical user), **p95**, **p99** (tail: your most-affected users, often the most valuable ones with the biggest carts). Percentiles cannot be averaged across instances; compute them from **histograms**.

13.2 Metrics in depth

Metric types (Prometheus model)

Type	Behavior	Example	Query with
Counter	Only goes up (resets to 0 on restart)	`http_requests_total`, `errors_total`	`rate()`, `increase()`: never graph a raw counter
Gauge	Goes up and down	`memory_bytes`, `queue_depth`, `temperature`	Directly, `avg_over_time()`, `max()`
Histogram	Counts observations into buckets (`_bucket{le="0.5"}`), plus `_sum` and `_count`	Request duration, response size	`histogram_quantile()`; aggregatable across instances
Summary	Client-side quantiles	Rarely used now	Cannot aggregate across instances: prefer histograms

Native histograms (Prometheus 2.40+, stable in 3.x) store high-resolution buckets automatically; classic histograms need sensible bucket boundaries around your SLO (e.g. 0.1, 0.25, 0.5, 1, 2.5 s).

Labels and cardinality

Each unique combination of metric name + label values is a separate **time series**. `http_requests_total{method, route, status}` with 5 methods × 50 routes × 10 statuses = 2,500 series per instance: fine. Adding `user_id` or `request_id` creates millions: **cardinality explosion**, which kills Prometheus memory and makes managed services expensive.

Rules: labels have bounded values (route template `/orders/{id}`, never the raw URL); never put IDs, emails, timestamps or error messages in labels; put high-cardinality detail in logs/traces.

Naming conventions

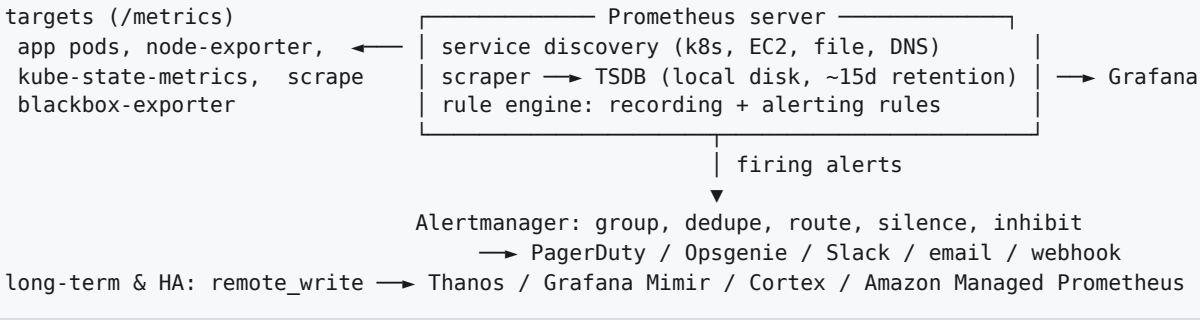
`<namespace>_<name>_<unit>_<suffix>`: `http_server_request_duration_seconds`, `shop_orders_created_total`. Base units (seconds, bytes), `_total` for counters, snake_case.

Pull vs push

	Pull (Prometheus)	Push (StatsD, CloudWatch PutMetricData, OTLP push)
How	Server scrapes `/metrics` endpoints every N seconds	Apps send metrics to a collector
Pros	Server controls rate; "target down" is itself a signal (`up == 0`); simple to debug (curl `/metrics`)	Works for short-lived jobs and behind firewalls
Short-lived jobs	Pushgateway (only for batch jobs)	Native

13.3 Prometheus

Architecture



Common exporters

Exporter	Exposes
node-exporter	Linux host metrics: CPU, memory, disk, filesystem, network
kube-state-metrics	Kubernetes object state: deployments, pod status, restarts, replicas
cAdvisor (in kubelet)	Per-container CPU/memory/network
blackbox-exporter	Probes from outside: HTTP, TCP, ICMP, DNS, TLS certificate expiry
postgres/mysql/redis exporters	Database internals
CloudWatch exporter / YACE	AWS service metrics into Prometheus

Configuration

```
# prometheus.yml
global:
  scrape_interval: 30s
  evaluation_interval: 30s
  external_labels:
    cluster: shop-prod                  # identifies this Prometheus in federated/remote storage

rule_files:
  - rules/*.yml

alerting:
  alertmanagers:
    - static_configs:
        - targets: ["alertmanager:9093"]

scrape_configs:
  - job_name: prometheus
    static_configs:
      - targets: ["localhost:9090"]

  - job_name: node
    static_configs:
      - targets: ["10.0.1.10:9100", "10.0.1.11:9100"]
        labels:
          env: prod

  - job_name: ec2-nodes                 # discover EC2 instances by tag
    ec2_sd_configs:
      - region: ap-south-1
        port: 9100
        filters:
          - name: tag:Monitoring
            values: ["enabled"]
    relabel_configs:
      - source_labels: [__meta_ec2_tag_Name]
        target_label: instance_name

  - job_name: blackbox-http             # check the public site from outside
    metrics_path: /probe
    params:
      module: [http_2xx]
    static_configs:
      - targets: ["https://shop.example.com/healthz"]
    relabel_configs:
      - source_labels: [__address__]
        target_label: __param_target
      - source_labels: [__param_target]
        target_label: instance
      - target_label: __address__
        replacement: blackbox-exporter:9115
```

Relabeling rewrites labels before scraping (`relabel_configs` : choose targets, set labels from discovery metadata) or after (`metric_relabel_configs` : drop expensive series). It is the main tool for controlling cost and cardinality.

On Kubernetes: kube-prometheus-stack

The Helm chart installs the **Prometheus Operator**, Prometheus, Alertmanager, Grafana, node-exporter, kube-state-metrics and default dashboards/alerts. You declare scrape targets and rules as Kubernetes objects:

```yaml
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: api
  namespace: shop
  labels:
    release: monitoring                  # must match the operator's serviceMonitorSelector
spec:
  selector:
    matchLabels:
      app.kubernetes.io/name: api        # Services with this label
  endpoints:
    - port: http                         # named Service port
      path: /metrics
      interval: 30s
```

The most common "my target isn't scraped" cause: the `release` label (selector) doesn't match, or the Service port name is wrong. Check **Status → Targets** in the Prometheus UI.

Scaling and long-term storage

One Prometheus handles millions of active series but keeps data locally (default retention 15 days) and is a single point of failure. Production patterns: **two identical replicas** (HA; Alertmanager deduplicates) + `remote_write` to **Thanos**, **Grafana Mimir**, or **Amazon Managed Service for Prometheus** for long retention and a global view. Grafana **Alloy** or the OpenTelemetry Collector can scrape and forward instead of a full Prometheus ("agent mode").

13.4 PromQL in depth

Building blocks

Concept	Example	Meaning
Instant vector	`up`	Latest value of every series named `up`
Label matchers	`http_requests_total{job="api", status=~"5..", method!="GET"}`	`=`, `!=`, `=~` regex, `!~`
Range vector	`http_requests_total[5m]`	All samples in the last 5 minutes (input to functions)
Offset	`rate(http_requests_total[5m] offset 1w)`	Same time last week
Scalar math	`node_memory_MemAvailable_bytes / 1024 / 1024`	MiB

Essential functions

Function	Use
`rate(counter[5m])`	Per-second average increase over 5 min; handles counter resets. **Use for alerts and graphs**
`irate(counter[5m])`	Rate from the last two samples: spiky, for fast-moving graphs only
`increase(counter[1h])`	Total increase over the window (`rate × seconds`)
`sum by (label) (...)` / `without (...)`	Aggregate, keeping or dropping labels. Also `avg`, `min`, `max`, `count`, `topk(5, ...)`, `quantile`

Function	Use
`histogram_quantile(0.99, sum by (le) (rate(x_bucket[5m])))`	p99 from a histogram (keep `le` in the `by`)
`avg_over_time(gauge[1h])`, `max_over_time`, `quantile_over_time`	Aggregate one series over time
`absent(up{job="api"})`	Fires when the series doesn't exist (target disappeared completely)
`predict_linear(gauge[6h], 4*3600)`	Linear forecast (disk full in 4 hours?)
`changes()`, `resets()`, `delta()`, `deriv()`	Change detection on gauges
`label_replace()`, `clamp_min()`	Label and value manipulation

Rule: `rate` first, then `sum`. Never `rate(sum(...))`, because summing counters breaks reset detection. Range window ≥ 4 × scrape interval (30s scrape → `[2m]` minimum, `[5m]` common).

Query library (memorize the patterns)

```
# Request rate per service (RED: Rate)
sum by (service) (rate(http_requests_total[5m]))

# Error ratio (RED: Errors): 5xx / all
sum by (service) (rate(http_requests_total{status=~"5.."}[5m]))
  /
sum by (service) (rate(http_requests_total[5m]))

# p95 and p99 latency (RED: Duration)
histogram_quantile(0.95,
  sum by (le, service) (rate(http_request_duration_seconds_bucket[5m])))

# Share of requests faster than 300 ms (latency SLI); needs a 0.3 bucket
sum(rate(http_request_duration_seconds_bucket{le="0.3"}[5m]))
  /
sum(rate(http_request_duration_seconds_count[5m]))

# Targets that are down
up == 0

# CPU usage % per node (USE: Utilization)
100 * (1 - avg by (instance) (rate(node_cpu_seconds_total{mode="idle"}[5m])))

# Memory used %
100 * (1 - node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes)

# Filesystem will be full within 4 hours
predict_linear(node_filesystem_avail_bytes{fstype!~"tmpfs|overlay"}[6h], 4 * 3600) < 0

# Pods restarting in the last hour
increase(kube_pod_container_status_restarts_total[1h]) > 3

# Deployments without all replicas available
kube_deployment_status_replicas_available < kube_deployment_spec_replicas

# Container memory as % of its limit
sum by (namespace, pod, container) (container_memory_working_set_bytes{container!=""})
  /
sum by (namespace, pod, container) (kube_pod_container_resource_limits{resource="memory"})

# CPU throttling ratio per container
sum by (pod) (rate(container_cpu_cfs_throttled_periods_total[5m]))
  /
sum by (pod) (rate(container_cpu_cfs_periods_total[5m]))

# Top 5 pods by CPU
topk(5, sum by (pod) (rate(container_cpu_usage_seconds_total{container!=""}[5m])))

# Traffic now vs one week ago (ratio)
sum(rate(http_requests_total[5m])) / sum(rate(http_requests_total[5m] offset 1w))

# TLS certificate expires within 14 days (blackbox exporter)
probe_ssl_earliest_cert_expiry - time() < 14 * 86400
```

Vector matching (joins)

Binary operators match series with **identical label sets**. When they differ, use `on(...)` / `ignoring(...)`, and `group_left` / `group_right` for many-to-one:

```
# Add the "team" label from a metadata metric to the error rate
sum by (service) (rate(http_requests_total{status=~"5.."}[5m]))
  * on (service) group_left (team)
service_owner_info
```

13.5 Instrumenting an application

```
"""app.py: minimal HTTP service exposing RED metrics for Prometheus."""
import random
import time
from http.server import BaseHTTPRequestHandler, HTTPServer

from prometheus_client import Counter, Histogram, start_http_server

REQUESTS = Counter(
    "http_requests_total", "HTTP requests", ["method", "route", "status"]
)
LATENCY = Histogram(
    "http_request_duration_seconds", "Request duration", ["route"],
    buckets=(0.05, 0.1, 0.25, 0.3, 0.5, 1.0, 2.5, 5.0),
)


class Handler(BaseHTTPRequestHandler):
    def do_GET(self):
        route = "/orders" if self.path.startswith("/orders") else "other"
        start = time.perf_counter()
        time.sleep(random.uniform(0.01, 0.4))            # simulated work
        status = 500 if random.random() < 0.02 else 200  # 2% errors
        self.send_response(status)
        self.end_headers()
        self.wfile.write(b"ok\n")
        LATENCY.labels(route=route).observe(time.perf_counter() - start)
        REQUESTS.labels(method="GET", route=route, status=str(status)).inc()

    def log_message(self, *args):                        # silence default access log
        pass


if __name__ == "__main__":
    start_http_server(9000)                              # metrics on :9000/metrics
    HTTPServer(("0.0.0.0", 8000), Handler).serve_forever()
```

Run: `pip install prometheus-client`, `python3 app.py`, send traffic (`for i in $(seq 200); do curl -s localhost:8000/orders >/dev/null; done`), then `curl -s localhost:9000/metrics | grep http_`. In real services use the framework integrations (Flask/FastAPI middleware, Spring Boot Actuator/Micrometer, Go `promhttp`) or **OpenTelemetry** SDKs (13.9).

13.6 Alerting

Alerting principles (what separates good on-call from burnout)

1. **Page only on symptoms that hurt users now** or will very soon (SLO burn, site down, data loss risk). Everything else is a **ticket** or dashboard.
2. Every page must be **actionable**, and must link a **runbook**.
3. Alert on **rates and ratios over windows**, not single samples; use `for:` to ignore blips.
4. Fewer, better alerts. A noisy alert trains people to ignore the real one (**alert fatigue**).
5. Review alerts after every incident and regularly: delete ones nobody acts on.

Severity	Response	Example
`critical` (page)	Wake someone up, immediately	Error budget burning 14× fast, checkout down
`warning` (ticket)	Next business day	Disk full in 3 days, certificate expires in 14 days
`info`	No notification (dashboards)	Deployment happened

Recording and alerting rules

Recording rules precompute expensive queries into new series (faster dashboards, consistent SLO math). Naming: `level:metric:operations`.

```
# rules/api.yml
groups:
  - name: api-recording
    interval: 30s
    rules:
      - record: service:http_requests:rate5m
        expr: sum by (service) (rate(http_requests_total[5m]))
      - record: service:http_errors:ratio_rate5m
        expr: |
          sum by (service) (rate(http_requests_total{status=~"5.."}[5m]))
            /
          sum by (service) (rate(http_requests_total[5m]))

  - name: api-alerts
    rules:
      - alert: ApiHighErrorRate
        expr: service:http_errors:ratio_rate5m{service="api"} > 0.05
        for: 5m                           # must be true for 5 minutes before firing
        labels:
          severity: critical
          team: shop
        annotations:
          summary: "API error rate {{ $value | humanizePercentage }} (> 5%)"
          description: "5xx ratio above 5% for 5 minutes on {{ $labels.service }}."
          runbook_url: https://wiki.example.com/runbooks/api-high-error-rate
          dashboard: https://grafana.example.com/d/api-red

      - alert: TargetDown
        expr: up{job="api"} == 0
        for: 3m
        labels:
          severity: warning
        annotations:
          summary: "{{ $labels.instance }} is not being scraped"

      - alert: DiskWillFillIn4Hours
        expr: |
          predict_linear(node_filesystem_avail_bytes{fstype!~"tmpfs|overlay"}[6h], 4 * 3600) < 0
        for: 30m
        labels:
          severity: warning
        annotations:
          summary: "Filesystem {{ $labels.mountpoint }} on {{ $labels.instance }} fills in < 4h"

      - alert: CertificateExpiresSoon
        expr: probe_ssl_earliest_cert_expiry - time() < 14 * 86400
        for: 1h
        labels:
          severity: warning
        annotations:
          summary: "TLS certificate for {{ $labels.instance }} expires in < 14 days"
```

Alert states: **inactive → pending** (condition true, waiting for `for`) **→ firing**. Validate rules in CI: `promtool check rules rules/*.yml`; unit-test them with `promtool test rules`.

Alertmanager

Receives alerts and decides **who** gets notified, **how often**, and **in what groups**.

```
# alertmanager.yml
route:
  receiver: default-slack
  group_by: ["alertname", "service"]     # one notification per group, not per series
  group_wait: 30s                        # wait to collect alerts of a new group
  group_interval: 5m                     # wait before sending updates for a group
  repeat_interval: 4h                    # re-send if still firing
  routes:
    - matchers: ['severity="critical"']
      receiver: pagerduty-shop
      continue: true                     # also evaluate the next routes
    - matchers: ['severity="critical"']
      receiver: shop-slack
    - matchers: ['severity="warning"']
      receiver: shop-slack

inhibit_rules:                           # suppress symptoms when the cause is known
  - source_matchers: ['alertname="ClusterUnreachable"']
    target_matchers: ['severity=~"warning|critical"']
    equal: ["cluster"]

receivers:
  - name: default-slack
    slack_configs:
      - api_url_file: /etc/alertmanager/secrets/slack-webhook   # secret from a file
        channel: "#alerts"
        send_resolved: true
  - name: shop-slack
    slack_configs:
      - api_url_file: /etc/alertmanager/secrets/slack-webhook
        channel: "#shop-alerts"
        send_resolved: true
  - name: pagerduty-shop
    pagerduty_configs:
      - routing_key_file: /etc/alertmanager/secrets/pagerduty-key
```

Feature	Purpose
Grouping	50 pods failing = one notification
Deduplication	HA Prometheus pairs send the same alert twice; Alertmanager sends once
Silences	Mute matching alerts for a time window (planned maintenance): `amtool silence add alertname=TargetDown --duration=2h --comment="patching"`
Inhibition	Mute dependent alerts when a root-cause alert fires

Validate: `amtool check-config alertmanager.yml`. Webhook URLs and keys come from files mounted from a secret store, never written in the config in Git.

13.7 Grafana

Concept	Meaning
Data source	Prometheus, Loki, Tempo, CloudWatch, Elasticsearch, PostgreSQL…

Concept	Meaning
Dashboard	Panels (time series, stat, gauge, table, heatmap, logs) built from queries
Variables	Drop-downs (`$namespace` , `$service`) from queries like `label_values(up, namespace)`
Annotations	Mark deploys/incidents on graphs
Alerting	Grafana can also evaluate alert rules (unified alerting)
Explore	Ad-hoc querying; jump from metrics to logs to traces

Dashboard design:

- One **overview** per service following RED/golden signals: top row = SLO status, request rate, error ratio, p95/p99 latency; below = saturation (CPU, memory, pool usage); bottom = dependencies.
- Consistent units and colors; titles say what "good" looks like.
- Link to drill-down dashboards and runbooks; avoid 40-panel "wall of graphs".

Dashboards as code: provisioning files (YAML + JSON) in Git, the Grafana Terraform provider, or the Grafana Operator. With kube-prometheus-stack, a ConfigMap labeled `grafana_dashboard: "1"` is loaded automatically. Don't hand-edit production dashboards without saving them back to Git.

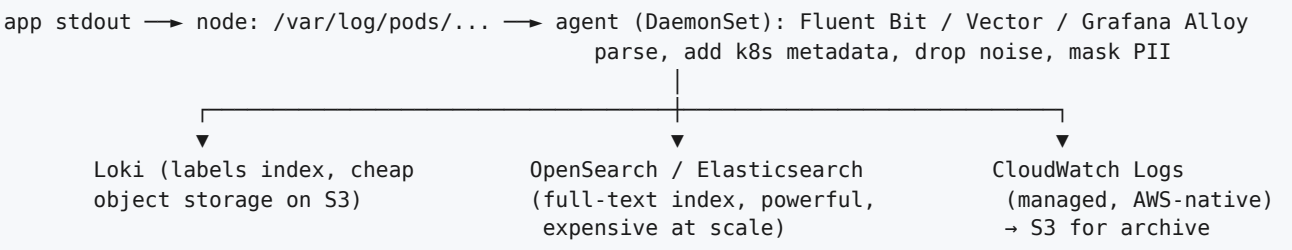
13.8 Logging

Good logs

Practice	Why
Structured (JSON) with consistent fields: `timestamp` , `level` , `service` , `env` , `version` , `trace_id` , `message`	Machine-searchable, joinable with traces
Correct levels: `DEBUG` (off in prod), `INFO` (business events), `WARN` (unexpected but handled), `ERROR` (failed operation), `FATAL`	Filtering and alerting
Log to **stdout/stderr** in containers	The platform collects them; no log files inside containers
Never log secrets or personal data (passwords, tokens, card numbers, full emails); mask at source	Security and privacy law (GDPR, DPDP Act in India)
Include context (order ID, user ID hash, route) but not huge payloads	Debuggable and affordable
Sample or rate-limit noisy logs	Cost

```
{"timestamp":"2026-10-18T09:14:03.512Z","level":"ERROR","service":"api","env":"prod",
 "version":"1.4.2","trace_id":"4bf92f3577b34da6a3ce929d0e0e4736","route":"/orders",
 "message":"payment provider timeout","duration_ms":3002}
```

Log pipeline



Backend	Trade-off
Grafana Loki	Indexes only labels; stores compressed chunks on S3: cheap, scales; queries scan text (keep label sets small)
OpenSearch / ELK	Full-text indexing: fast arbitrary search, costly storage/compute
CloudWatch Logs	Zero-ops on AWS, Logs Insights queries; ingestion cost adds up: set retention per log group
Splunk / Datadog	Commercial, feature-rich, priced by volume

Retention and cost: decide retention per class (debug 7 days, app 30 days, audit/security 1+ years in cheap storage). Ingestion volume is the cost driver: drop health-check and debug noise at the agent.

LogQL (Loki) essentials

```
{namespace="shop", app="api"} |= "error"                          # filter lines containing "error"
{namespace="shop", app="api"} | json | level="ERROR" | route="/orders"
sum by (route) (count_over_time({app="api"} | json | level="ERROR" [5m]))   # errors/5m by route
{app="api"} | json | duration_ms > 1000 | line_format "{{.route}} {{.duration_ms}}ms"
```

13.9 Distributed tracing and OpenTelemetry

```
trace 4bf92f35... (one user request)
├── span: GET /checkout                api-gateway    480 ms
│   ├── span: POST /orders             orders-svc     430 ms
│   │   ├── span: SELECT orders        postgres        12 ms
│   │   └── span: POST /charge         payments-svc   400 ms  ◄── the slow part
│   └── span: GET /cart                cart-svc        25 ms
```

Term	Meaning
Trace	All spans of one request, sharing a **trace ID**
Span	One operation: name, start, duration, status, attributes (`http.route` , `db.system`), parent span
Context propagation	Passing trace/span IDs between services in headers: W3C `traceparent` standard
Sampling	Keep a fraction of traces: head-based (decide at start, e.g. 10%) or **tail-based** (decide after completion: keep all errors and slow ones) in the Collector

OpenTelemetry (OTel)

The CNCF **vendor-neutral standard** for metrics, logs and traces: APIs/SDKs for every major language, **auto-instrumentation** (Java agent, Python `opentelemetry-instrument` , Kubernetes operator injection), the **OTLP** protocol, and the **Collector**. Instrument once, send anywhere (Tempo, Jaeger, X-Ray, Datadog, Prometheus).

```
# otel-collector.yaml: receive OTLP, batch, tail-sample, export traces and metrics
receivers:
  otlp:
    protocols:
      grpc:
        endpoint: 0.0.0.0:4317
      http:
        endpoint: 0.0.0.0:4318
processors:
  memory_limiter:
    check_interval: 1s
    limit_percentage: 80
  k8sattributes: {}                     # add pod/namespace/deployment attributes
  tail_sampling:
    decision_wait: 10s
    policies:
      - name: keep-errors
        type: status_code
        status_code:
          status_codes: [ERROR]
      - name: keep-slow
        type: latency
        latency:
          threshold_ms: 1000
      - name: sample-rest
        type: probabilistic
        probabilistic:
          sampling_percentage: 10
  batch: {}
exporters:
  otlp/tempo:
    endpoint: tempo.monitoring:4317
    tls:
      insecure: true                    # in-cluster only; use TLS across networks
  prometheusremotewrite:
    endpoint: http://mimir.monitoring/api/v1/push
service:
  pipelines:
    traces:
      receivers: [otlp]
      processors: [memory_limiter, k8sattributes, tail_sampling, batch]
      exporters: [otlp/tempo]
    metrics:
      receivers: [otlp]
      processors: [memory_limiter, k8sattributes, batch]
      exporters: [prometheusremotewrite]
```

Python auto-instrumentation: `pip install opentelemetry-distro opentelemetry-exporter-otlp`, `opentelemetry-bootstrap -a install`, then run with `OTEL_SERVICE_NAME=api OTEL_EXPORTER_OTLP_ENDPOINT=http://otel-collector:4317 opentelemetry-instrument python app.py`.

13.10 AWS CloudWatch and native observability

Service	Use
CloudWatch Metrics	AWS services publish automatically (EC2 CPU, ALB `HTTPCode_Target_5XX_Count`, `TargetResponseTime`, RDS `FreeStorageSpace`, Lambda `Errors` / `Duration`). Standard resolution 1 min (EC2 basic: 5 min; detailed monitoring: 1 min)
CloudWatch agent	OS-level metrics (memory, disk: **not** collected by default on EC2) and log files
CloudWatch Alarms	Threshold or anomaly-detection alarms → SNS (email, chat, PagerDuty), Auto Scaling, EC2 actions. **Composite alarms** combine alarms to cut noise

Service	Use
CloudWatch Logs	Log groups/streams, retention, metric filters (turn log patterns into metrics), subscription filters (stream to Lambda/Firehose/OpenSearch)
Logs Insights	Query language over logs
Container Insights / Application Signals	EKS/ECS metrics and logs; service-level SLOs
X-Ray / CloudWatch traces	Distributed tracing (accepts OpenTelemetry via ADOT)
Synthetics canaries	Scripted checks of URLs and user journeys
Amazon Managed Prometheus / Managed Grafana	Hosted Prometheus-compatible storage and Grafana
AWS Health, CloudTrail	Provider incidents; API audit trail (who changed what)

```
# Alarm: ALB target 5xx > 10 per minute for 3 consecutive minutes -> SNS topic
aws cloudwatch put-metric-alarm --alarm-name shop-alb-5xx \
  --namespace AWS/ApplicationELB --metric-name HTTPCode_Target_5XX_Count \
  --dimensions Name=LoadBalancer,Value=app/shop-alb/0123456789abcdef \
  --statistic Sum --period 60 --evaluation-periods 3 --threshold 10 \
  --comparison-operator GreaterThanThreshold --treat-missing-data notBreaching \
  --alarm-actions arn:aws:sns:ap-south-1:123456789012:shop-alerts
```

```
# Logs Insights: top error messages in the last hour
fields @timestamp, level, message
| filter level = "ERROR"
| stats count(*) as errors by message
| sort errors desc
| limit 20
```

Cost awareness: custom metrics, high-resolution metrics, log ingestion (per GB) and long retention cost money. Set **retention on every log group** (default is "never expire"), avoid per-request custom metrics, and use metric filters or EMF (Embedded Metric Format) instead of `PutMetricData` loops.

13.11 Site Reliability Engineering

SRE (from Google) is "what happens when you treat operations as a software problem". Core ideas:

Principle	In practice
Reliability is a **feature with a target**	100% is the wrong target: users can't tell 99.99% from 100%, and it costs exponentially more. Pick a target the business needs
Error budgets balance speed and stability	Budget left → ship faster; budget spent → focus on reliability
Eliminate **toil**	Manual, repetitive, automatable work that scales with load. Cap it (Google: under 50% of SRE time) and automate it
Monitoring on symptoms, with SLOs	Page on user impact, not on every CPU spike
Blameless postmortems	Fix systems and processes, not people

Principle	In practice
Simplicity, gradual rollouts, automation	Most outages come from changes: make changes small, progressive, reversible

SLI, SLO, SLA

Term	Definition	Example
SLI (indicator)	A measured ratio of good events to valid events	Successful (non-5xx) requests / all requests; requests faster than 300 ms / all requests
SLO (objective)	Target for an SLI over a window	99.9% of requests succeed over a rolling 30 days
SLA (agreement)	Contract with customers, with penalties (credits)	99.5% monthly availability or 10% credit. SLA is **looser** than the SLO so you're warned first
Error budget	`1 − SLO`: the failure you're allowed	0.1% of requests may fail in 30 days

Good SLIs are measured as close to the user as possible (load balancer or client, not CPU), and cover what users care about: availability, latency, correctness, freshness (data pipelines), durability (storage).

Error budget math (30-day window)

SLO	Allowed bad time per 30 days	Per day
99%	7 h 12 min	14 min 24 s
99.5%	3 h 36 min	7 min 12 s
99.9% ("three nines")	**43 min 12 s**	1 min 26 s
99.95%	21 min 36 s	43 s
99.99%	4 min 19 s	8.6 s
99.999%	26 s	0.9 s

Request-based example: 10,000,000 requests per 30 days at 99.9% → budget = **10,000 failed requests**. If an incident fails 4,000 requests, 40% of the budget is gone.

Dependencies multiply: a service calling three dependencies each at 99.9% (in series) can be at most ~99.7% available. You can't promise more than your critical dependencies allow, unless you add redundancy or graceful degradation.

Error budget policy (agreed in writing with product owners)

- Budget remaining: releases proceed normally.
- Budget below 25%: extra review for risky changes, prioritize reliability items.
- Budget exhausted: **freeze feature releases** (except fixes and security) until the SLO recovers; postmortem required for any incident consuming more than 20% of the budget.

Alerting on SLOs: multi-window, multi-burn-rate

Burn rate = how fast the budget is consumed relative to "exactly on budget". Burn rate 1 uses the whole budget in 30 days; burn rate 14.4 uses it in about 2 days.

Severity	Burn rate	Long window	Short window	Budget consumed when it fires
Page	14.4	1 h	5 min	2%

Severity	Burn rate	Long window	Short window	Budget consumed when it fires
Page	6	6 h	30 min	5%
Ticket	1	3 days	6 h	10%

Both windows must exceed the threshold: the long window proves it's significant, the short window proves it's **still happening** (so the alert resolves quickly after recovery).

```
# SLO 99.9% availability -> error budget 0.001
groups:
  - name: api-slo
    rules:
      - record: service:sli_errors:ratio_rate5m
        expr: |
          sum by (service) (rate(http_requests_total{status=~"5.."}[5m]))
            / sum by (service) (rate(http_requests_total[5m]))
      - record: service:sli_errors:ratio_rate30m
        expr: |
          sum by (service) (rate(http_requests_total{status=~"5.."}[30m]))
            / sum by (service) (rate(http_requests_total[30m]))
      - record: service:sli_errors:ratio_rate1h
        expr: |
          sum by (service) (rate(http_requests_total{status=~"5.."}[1h]))
            / sum by (service) (rate(http_requests_total[1h]))
      - record: service:sli_errors:ratio_rate6h
        expr: |
          sum by (service) (rate(http_requests_total{status=~"5.."}[6h]))
            / sum by (service) (rate(http_requests_total[6h]))
      - alert: ApiErrorBudgetFastBurn
        expr: |
          (
            service:sli_errors:ratio_rate1h{service="api"} > (14.4 * 0.001)
            and
            service:sli_errors:ratio_rate5m{service="api"} > (14.4 * 0.001)
          )
          or
          (
            service:sli_errors:ratio_rate6h{service="api"} > (6 * 0.001)
            and
            service:sli_errors:ratio_rate30m{service="api"} > (6 * 0.001)
          )
        labels:
          severity: critical
        annotations:
          summary: "API is burning its 30-day error budget too fast"
          runbook_url: https://wiki.example.com/runbooks/api-error-budget
```

Tools generate these rules from a short SLO definition: **Sloth**, **Pyrra**, OpenSLO spec, Grafana SLO, CloudWatch Application Signals SLOs.

13.12 Incident management

Severity levels (example)

Sev	Definition	Response
SEV1	Critical user-facing outage or data loss/security breach	All hands, incident commander, status page, exec updates every 30 min
SEV2	Major degradation, key feature broken for many users	On-call + team, updates every hour

Sev	Definition	Response
SEV3	Minor impact, workaround exists	Normal working hours
SEV4	No user impact	Ticket

Roles

Role	Responsibility
Incident Commander (IC)	Coordinates, decides, delegates. Does **not** debug
Operations / technical lead	Investigates and applies fixes
Communications lead	Status page, stakeholders, support teams
Scribe	Timeline of actions and findings
Subject-matter experts	Pulled in as needed

Lifecycle

```
Detect ⟶ Triage (severity, declare, open channel) ⟶ Mitigate (stop the bleeding) ⟶
Resolve (root fix) ⟶ Postmortem (within ~5 days) ⟶ Follow-up actions tracked to completion
```

Mitigate first, investigate later: roll back the last deploy, shift traffic away from a bad region/AZ, scale up, disable a feature flag, fail over. Root cause can wait until users are fine.

Key metrics: MTTD (time to detect), MTTA (to acknowledge), **MTTR** (to restore), incident count by severity, % of incidents detected by monitoring (vs by customers).

On-call that works

- Rotations with primary and secondary, handover notes, fair schedules across time zones (follow-the-sun when possible).
- Every alert has a **runbook**: what it means, how to check impact, mitigation steps, escalation contacts, dashboards.
- Track page volume; more than ~2 incidents per 12-hour shift means the system or alerts need work.
- Tools: PagerDuty, Opsgenie, Grafana OnCall/IRM, incident.io, FireHydrant.

Runbook template

```
Alert: ApiHighErrorRate (critical)
Meaning:     >5% of API requests return 5xx for 5 minutes. Users see failed checkouts.
Check:       Grafana "API RED" dashboard; recent deploys (Argo CD history); dependency status.
Mitigate:    1. If a deploy happened in the last 30 min: roll back (argocd app rollback shop-api).
             2. If one AZ/pod set is bad: cordon/drain or scale; check ALB target health.
             3. If a dependency (payments) is failing: enable feature flag "payments-degraded".
Escalate:    After 15 min without mitigation: page shop secondary + payments on-call.
Verify:      Error ratio < 1% for 10 min; synthetic checkout canary green.
```

Blameless postmortem template

Section	Content
Summary	What happened, impact in numbers (duration, users/requests affected, revenue, budget consumed)
Timeline	UTC timestamps: trigger, detection, key decisions, mitigation, resolution

Section	Content
Root cause(s) and contributing factors	Technical and process causes (use "5 whys"; there is rarely one root cause)
Detection	How we found out; could monitoring have caught it sooner?
What went well / what went badly / where we got lucky	Honest learning
Action items	Specific, owned, dated, prioritized (prevent, detect, mitigate)

"Blameless" means people describe what they did and why it made sense at the time, without fear. Human error is a starting point for asking **why the system allowed it**.

13.13 Capacity, load testing and chaos engineering

Practice	How
Capacity planning	Track saturation trends (CPU, memory, connections, queue depth) and traffic growth; forecast (`predict_linear` , seasonality); keep headroom (e.g. survive losing one AZ at peak)
Load testing	k6, Locust, JMeter, Gatling: find the breaking point and the latency curve before users do; run in a production-like environment; test autoscaling
Chaos engineering	Inject failures deliberately to prove resilience: kill pods/nodes, add latency, fail an AZ, exhaust CPU. Tools: Chaos Mesh, LitmusChaos, AWS Fault Injection Service. Start with a **hypothesis**, small blast radius, in staging, with a stop condition
Game days	Scheduled drills where the team practices incident response (and restores from backup)

```
// k6 load test: ramp to 200 virtual users, fail the run if SLO-like thresholds break
import http from "k6/http";
import { check, sleep } from "k6";

export const options = {
  stages: [
    { duration: "2m", target: 50 },
    { duration: "5m", target: 200 },
    { duration: "2m", target: 0 },
  ],
  thresholds: {
    http_req_failed: ["rate<0.001"],     // < 0.1% errors
    http_req_duration: ["p(95)<300"],    // p95 < 300 ms
  },
};

export default function () {
  const res = http.get("https://staging.shop.example.com/api/products");
  check(res, { "status is 200": (r) => r.status === 200 });
  sleep(1);
}
```

13.14 A production monitoring stack (reference design)

```
                          ┌──────────── Grafana (dashboards, Explore, SLOs) ────────────┐
                          │                    │                    │                   │
Metrics:  apps ─OTLP/scrape─► Prometheus (x2, HA) ─remote_write─► Mimir / AMP (long-term)
Logs:     stdout ─► Fluent Bit / Alloy DaemonSet ─► Loki (S3)        or CloudWatch Logs
Traces:   OTel SDK ─► OTel Collector (tail sampling) ─► Tempo (S3)   or X-Ray
Probes:   blackbox exporter + synthetic canaries (outside-in, from several regions)
Alerts:   Prometheus rules (SLO burn, golden signals) ─► Alertmanager ─► PagerDuty / Slack
Changes:  Argo CD / CI deploy events ─► Grafana annotations
```

Must-haves before go-live: golden-signal dashboard per service, SLO + burn-rate alerts, runbooks linked from alerts, external synthetic check, log retention policy, trace IDs in logs, on-call rotation, alert routing tested end-to-end (fire a test alert).

Who watches the watcher? A **dead man's switch**: an always-firing `Watchdog` alert sent to an external service (Healthchecks.io, PagerDuty heartbeat, Cronitor) that pages you when it **stops** arriving. kube-prometheus-stack ships this alert by default.

13.15 Troubleshooting the monitoring system

Symptom	Check
Target missing in Prometheus	Status → Targets / Service Discovery; ServiceMonitor `release` label; Service port name; NetworkPolicy blocking the scrape
Target `DOWN`	`curl` the `/metrics` endpoint from the Prometheus pod; TLS/auth; wrong port
Query returns nothing	Label names/values (use Explore autocomplete); `rate()` window shorter than 2 scrapes; metric renamed
Graph spikes or drops at restarts	Using `irate` or raw counters; use `rate`
Prometheus OOM / slow	Cardinality: `topk(10, count by (__name__) ({__name__=~".+"}))`, TSDB status page; drop series with `metric_relabel_configs`
Alert never fires	`promtool test rules`; alert pending (check `for`); Alertmanager routing (`amtool config routes test`); silences/inhibitions
Alert floods	Group by better labels, add `for`, inhibition, fix the threshold or delete the alert
No logs in Loki/CloudWatch	Agent DaemonSet running on that node? Parsing errors in agent logs; permissions (IAM for CloudWatch); rate limits on ingestion
Traces broken into pieces	Context not propagated (missing instrumentation on an HTTP client or queue); mixed propagation formats

13.16 Hands-on labs

#	Lab
1	Run Prometheus + node-exporter + Grafana with Docker Compose; add Prometheus as a data source; import the "Node Exporter Full" dashboard
2	Run the 13.5 app; scrape it; write the RED queries from 13.4
3	Add the recording and alerting rules from 13.6; validate with `promtool check rules`; trigger the error alert by raising the error rate

#	Lab
4	Add Alertmanager with a Slack/email receiver (webhook from a local file); test grouping and a silence
5	Add blackbox-exporter; monitor a public URL and its certificate expiry
6	On kind: install kube-prometheus-stack with Helm; add a ServiceMonitor for your app; break the label and debug
7	Add Loki + Alloy/Fluent Bit; query JSON logs with LogQL; build a logs panel next to metrics
8	Add OpenTelemetry auto-instrumentation and Tempo; follow one slow request from metric to trace to log
9	Define a 99.9% SLO; implement the burn-rate alerts from 13.11; calculate the remaining budget after a simulated incident
10	Run the k6 test against your app; record the breaking point; write a one-page postmortem for the "incident" you caused
11	(AWS, small cost: watch the free plan credits) CloudWatch agent on an EC2 instance, a CPU alarm to SNS email, a Logs Insights query; delete everything afterwards

13.17 Interview questions

1. Monitoring vs observability; metrics vs logs vs traces: when do you use each?
2. Explain the four golden signals, RED and USE.
3. Why are averages misleading? How do you compute p99 in Prometheus?
4. Counter vs gauge vs histogram vs summary.
5. What is cardinality and how do you prevent a cardinality explosion?
6. Explain Prometheus architecture and the pull model; how is a short-lived job monitored?
7. Write PromQL for request rate, error ratio and p95 latency per service.
8. Why `rate()` before `sum()`? `rate` vs `irate` vs `increase`?
9. How does Prometheus discover targets on Kubernetes (ServiceMonitor)? A target is missing: steps?
10. How do you make Prometheus highly available and keep a year of data?
11. Recording rules: why? Alerting rule states (`pending`, `firing`) and `for:`.
12. What makes a good alert? How do you fight alert fatigue?
13. Alertmanager grouping, inhibition, silences, routing.
14. Structured logging: what fields, and what must never be logged?
15. Loki vs Elasticsearch vs CloudWatch Logs trade-offs.
16. Explain a trace, spans, context propagation and sampling (head vs tail).
17. What is OpenTelemetry and why are companies standardizing on it?
18. Which CloudWatch metrics are not collected by default on EC2? How do you alarm on ALB 5xx?
19. SLI vs SLO vs SLA, with examples. How do you choose an SLO?
20. Calculate the monthly error budget for 99.9% and 99.95%.
21. What is burn-rate alerting and why multi-window?
22. What is an error budget policy? What happens when the budget is exhausted?
23. What is toil? Give examples you've automated.
24. Walk through how you'd run a SEV1 incident. What does the incident commander do?
25. What is a blameless postmortem? What sections does it have?
26. MTTD, MTTA, MTTR: how would you reduce each?
27. How do you monitor the monitoring system (dead man's switch)?
28. Load testing vs chaos engineering: how and when?
29. Design observability for a new microservice platform on EKS.

13.18 Self-check

- [ ] I can explain the signals, golden signals, RED and USE, and why percentiles matter.
- [ ] I can choose metric types and keep labels low-cardinality.
- [ ] I can configure Prometheus scraping (static, EC2, Kubernetes ServiceMonitor) and debug missing targets.
- [ ] I can write the PromQL query library in 13.4 from memory.
- [ ] I can write recording/alerting rules and an Alertmanager routing tree, and validate them with promtool/amtool.
- [ ] I can design a Grafana service dashboard and keep it as code.
- [ ] I can set up structured logging, a log pipeline and retention, and query with LogQL / Logs Insights.
- [ ] I can explain tracing and configure an OpenTelemetry Collector.
- [ ] I can define SLIs/SLOs, compute error budgets and implement burn-rate alerts.
- [ ] I can run an incident (roles, mitigation first) and write a blameless postmortem.

REMEMBER

- **SLI** = what you measure · **SLO** = your target · **SLA** = promise to customers.
- Alert on what users feel; every alert must be actionable.
- During an incident: **restore service first, investigate second**.